



Tracing AI Requests with OpenTelemetry

End-to-end visibility across distributed AI systems

An essential guide for engineers and SREs implementing observability in modern AI infrastructure

What Is OpenTelemetry?

An open-source **observability framework** under the Cloud Native Computing Foundation (CNCF) that:

Unified Standard

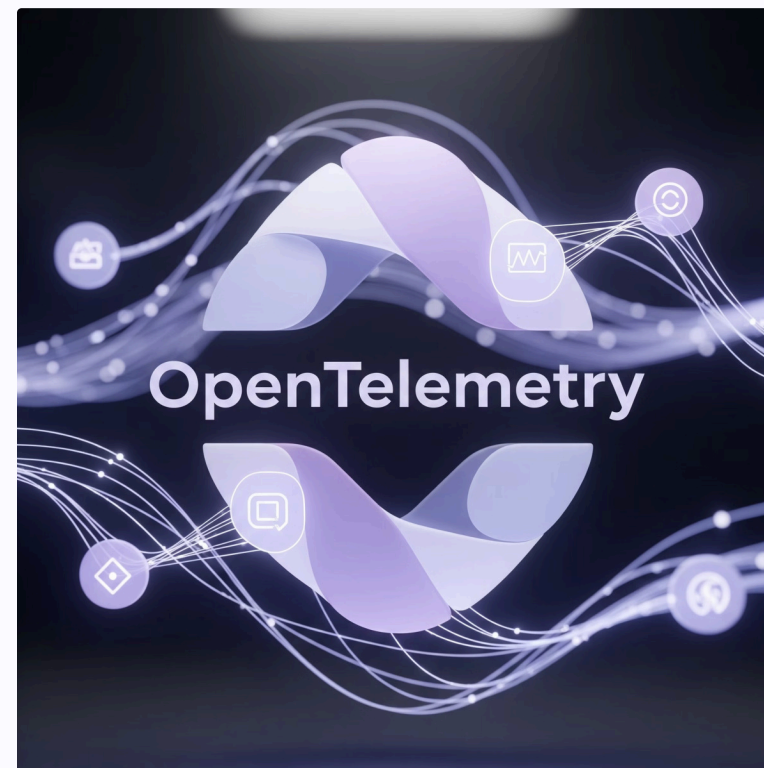
Collects **traces, metrics, logs** in a standardized format across your infrastructure

Ecosystem Integration

Works seamlessly with Prometheus, Grafana, Jaeger, Datadog, and other observability tools

Multi-Language Support

Provides SDKs for Python, Go, Java, C++, and many other languages



Why Tracing Matters for AI

Multi-Layer Complexity

AI requests traverse **multiple layers**: API → preprocessing → model inference → database → postprocessing

Hidden Failure Points

Performance issues and failures often hide deep in the pipeline, making traditional debugging insufficient

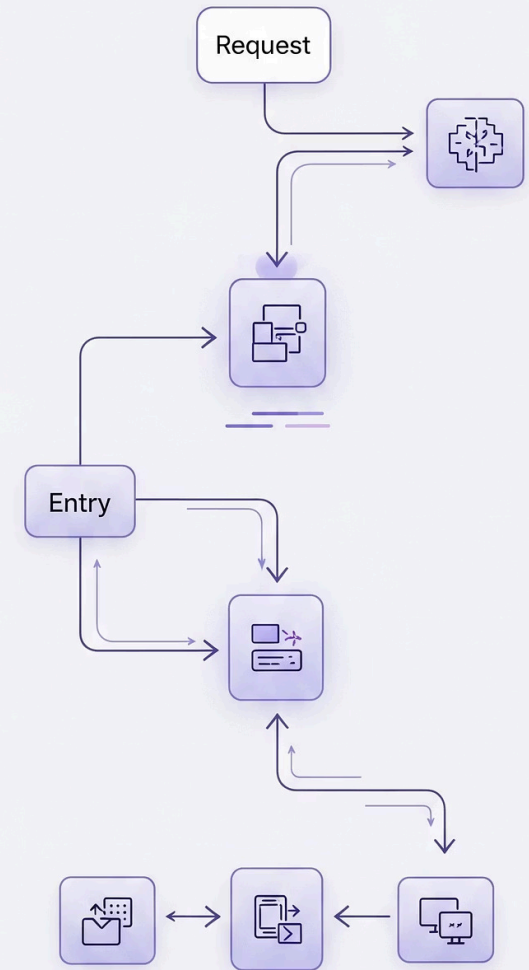
Root-Cause Visibility

Tracing provides **end-to-end visibility** across service boundaries, enabling faster issue resolution

Tail Latency Detection

Essential for debugging **p95/p99 latency outliers** that affect critical user experiences

Complex AI request



Core Tracing Concepts

Trace

Complete journey of a request across all services from initiation to completion

Span

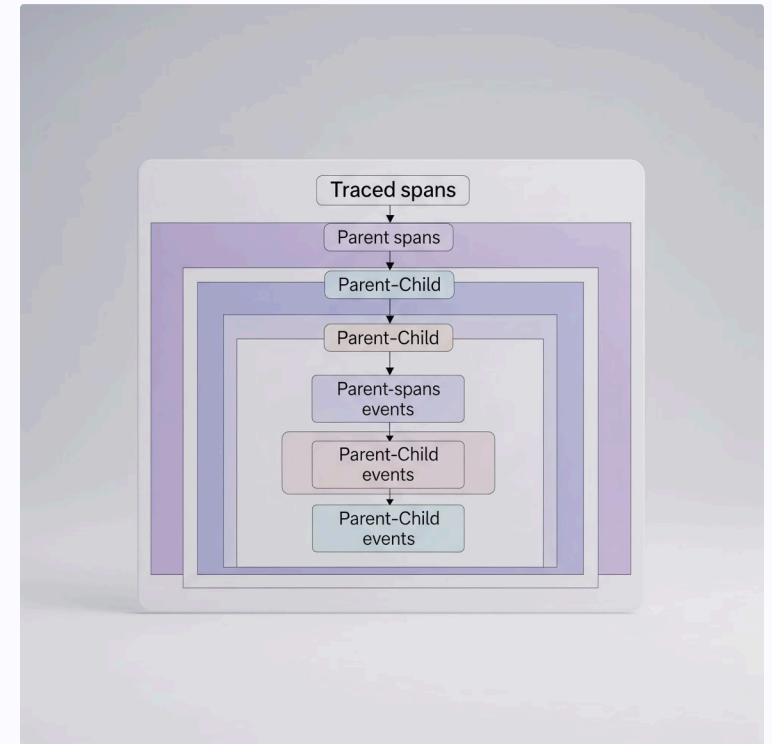
Individual operation within a trace (e.g., model inference, data preprocessing)

Context Propagation

Trace IDs passed between services to maintain request continuity

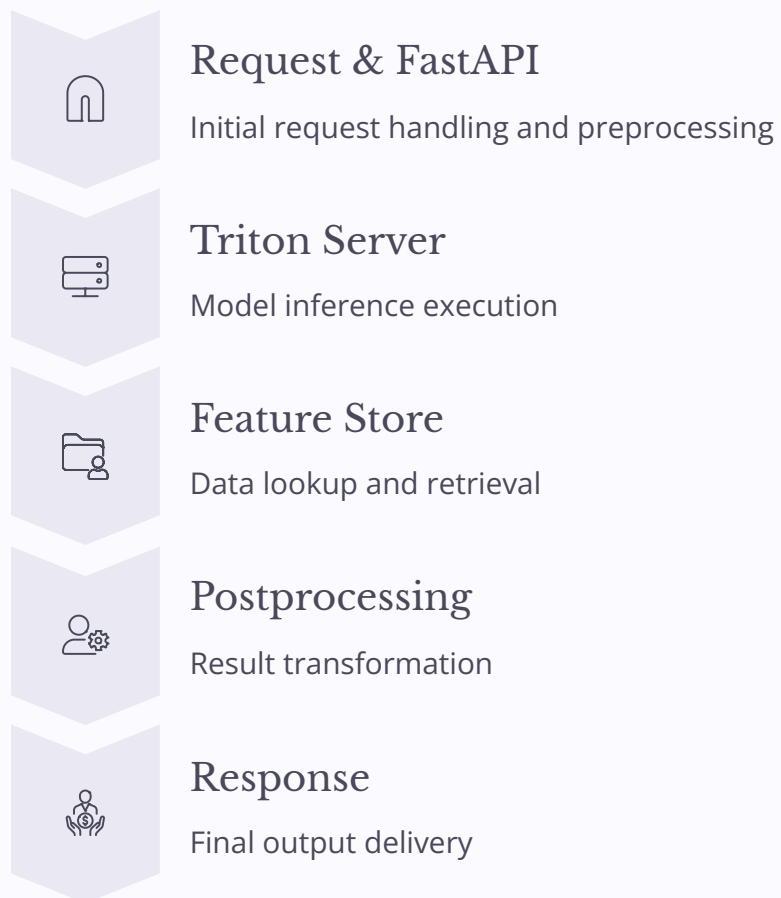
Attributes & Events

Metadata like model name, version, input size, and execution time



A trace contains multiple spans that represent operations within your AI pipeline. Each span captures timing data and contextual information.

AI Pipeline Example Trace



Each stage represents a [span](#) in the trace. The complete trace reveals **latency breakdown** across services, enabling precise bottleneck identification.

Instrumenting AI Services

Python SDK for AI Frameworks

Easily add tracing to FastAPI, Flask, TorchServe, and other AI serving frameworks:

```
from opentelemetry.instrumentation.fastapi import  
FastAPIInstrumentor
```

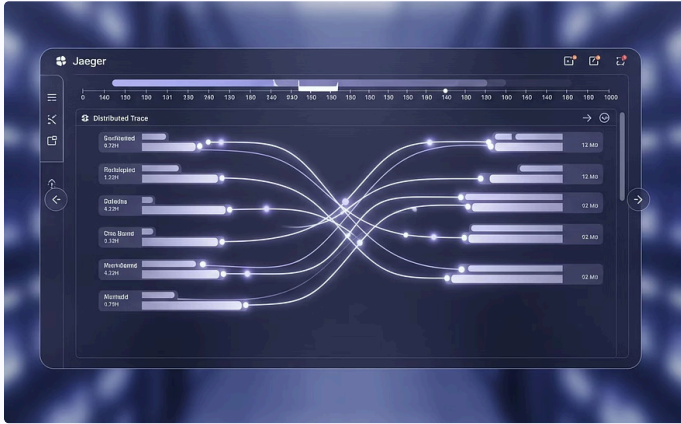
```
# One line to instrument your entire FastAPI app  
FastAPIInstrumentor().instrument_app(app)
```

Auto-instruments HTTP endpoints, database calls, and external dependencies

Key Implementation Steps

- Install OpenTelemetry SDK and instrumentation packages
- Configure trace exporter to your backend (Jaeger, etc.)
- Add auto-instrumentation for your framework
- Create custom spans for model-specific operations
- Attach trace IDs to logs for correlation

Exporters & Backends



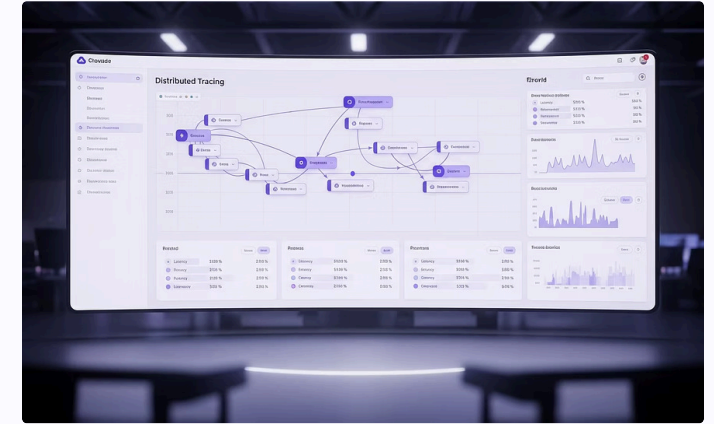
Open Source

Jaeger & Tempo provide powerful open-source tracing backends with visualization and query capabilities



Unified Observability

Grafana + Tempo + Loki combine traces, metrics, and logs for comprehensive visibility



Cloud-Native

AWS X-Ray, GCP Trace, Azure Monitor offer managed tracing solutions with cloud integration

All backends support the [OTLP \(OpenTelemetry Protocol\)](#) standard for receiving telemetry data, enabling flexibility in your observability stack.

Tracing GPU & Model Layers



Custom Instrumentation for ML Components

Model Execution Spans

Create spans capturing [inference time](#) for specific model components

Link Triton custom metrics to OpenTelemetry spans

Rich Context Attributes

Attach model version, batch size, GPU ID, and tensor shapes as span attributes

Infrastructure Correlation

Link spans to Prometheus metrics for correlating application performance with infrastructure metrics

Sampling Strategies

Balance observability depth with resource utilization by choosing the right sampling strategy:

100%

Always-on

Capture every single request

Pros: Complete visibility

Cons: Expensive at scale

1-10%

Probabilistic

Random sampling of traffic

Pros: Predictable overhead

Cons: May miss specific issues

p95+

Tail-based

Focus on slow/error cases

Pros: Captures problematic requests

Cons: Requires dynamic sampling implementation

Choose based on your [traffic volume](#), [infrastructure capacity](#), and [observability budget](#). Consider starting with higher sampling in development and reducing in production.

Benefits for AI Ops



Real-time Inference Debugging: Identify and resolve latency spikes in production model serving

Infrastructure Bottleneck Detection: Pinpoint feature store slowdowns or database congestion

Data Pipeline Correlation: Trace model drift issues to specific data source latency or quality problems

Accelerated Incident Response: Cut MTTR with precise root-cause insights across services

i According to our internal benchmarks, teams implementing comprehensive tracing reduced their mean time to resolution (MTTR) by 62% for complex AI pipeline issues.

Best Practices



End-to-End Propagation

Always propagate [trace context](#) across all service boundaries, including batch jobs and async workers



Consistent Tagging

Tag spans with standardized attributes for model, version, environment, and user/team identifiers



Strategic Sampling

Implement intelligent sampling to balance visibility with resource consumption and storage costs



Integrated Observability

Connect traces with logs and metrics for a complete observability solution that enables rapid debugging

Maintain a **consistent naming convention** for spans across services to ensure clear visualization and easier querying in your tracing backend.

Key Takeaways



OpenTelemetry Transforms AI Observability

Provides the [standard framework](#) for instrumenting modern AI pipelines

Delivers **end-to-end visibility** across distributed infrastructure components

Critical for debugging **tail latency issues** and identifying performance bottlenecks

Integrates with Prometheus and Grafana for [complete observability](#)

Start small, instrument critical paths first, and expand coverage incrementally for maximum impact.